

Seven Rules for Building Data-Intensive Systems

These rules were extracted from analyzing 37 reference implementations of concepts from *Designing Data-Intensive Applications*. Each one addresses a failure mode that emerged independently across multiple subsystems — storage engines, consensus protocols, replication layers, and derived data pipelines. They represent the gap between understanding a distributed systems concept and implementing it correctly.

None of these are novel. Kleppmann, Gray, Lamport, and others have written about all of them. What’s notable is how reliably they’re violated even when the implementer knows the theory. These rules are phrased as construction principles — things to do from the start — because retrofitting them is consistently harder than getting them right up front.

1. Verify integrity at every format boundary

When data changes format or location — WAL record to SSTable block, in-memory buffer to on-disk page, source table to derived index — verify its integrity on both sides of the transition. Don’t assume that because data was valid when written, it’s still valid when read back, compacted, or replicated.

What goes wrong: Teams protect the write path with checksums but skip verification during compaction, backup, replication, and read-back. Corruption enters through the unguarded transitions and propagates silently through every downstream consumer. By the time it’s detected (usually by an end user), the uncorrupted version has been garbage collected.

Specifically: - Checksum data at write time and verify at every subsequent read, including internal reads (compaction, merge, hint file generation). - Cover the full record — metadata, routing headers, and payload — not just the payload. A correct value with a corrupted key or sequence number is still corruption. - If you detect corruption, don’t silently skip the record and continue. At minimum, log it with enough context to investigate. Ideally, halt the operation and let the operator decide — silent data loss is worse than a noisy failure. - Design a corruption recovery path before you need one. “Restore from backup” is a recovery path. “We’ll figure it out” is not.

2. Test under the failure model you claim to tolerate

If your system claims partition tolerance, test it under network partitions. If it claims crash recovery, kill the process mid-operation and verify recovery. If it claims Byzantine fault tolerance, inject Byzantine faults. Testing exclusively under clean, synchronous conditions makes your safety claims unfalsifiable — a passing test suite tells you nothing about the failure modes that matter.

What goes wrong: Distributed protocol tests use synchronous, in-process message delivery with deterministic ordering. Every message arrives, in order, immediately. The tests pass — and they would pass even if the protocol had a fundamental safety violation that only manifests under reordering or delay. The team ships with false confidence.

Specifically: - Build a chaos layer into your test harness from day one: message reordering, duplication, delay, loss, and network partitions. These aren't edge cases — they're the entire reason you chose a distributed protocol. - For crash recovery, test the power-off scenario: `kill -9` the process at random points during write operations, then verify that recovery produces a consistent state. `flush()` without `fsync()` is not durable. A clean shutdown test is not a crash recovery test. - For consensus protocols, test leadership transitions under load. The steady-state single-leader path is the easy case. The hard cases are: leader failure during a write, split-brain during a network partition, and recovery after a partition heals. - Run long-duration soak tests, not just unit tests. Many distributed bugs require specific interleavings that only appear under sustained concurrent load.

3. Make concurrency assumptions explicit and enforced

If a component requires single-threaded access, enforce it — with a lock, an assertion, or a type-level constraint. If a component is thread-safe, document what “safe” means: can multiple threads read concurrently? Can one thread write while others read? Are compound operations (read-modify-write) atomic?

What goes wrong: Components silently assume single-threaded access. No lock, no assertion, no documentation. The assumption holds during initial development because there's only one caller. Then someone adds a background compaction thread, or a concurrent read path, or a connection pool. The component doesn't fail immediately — it produces subtly wrong results: torn reads, lost updates, corrupted internal state. These bugs are intermittent, hard to reproduce, and usually misdiagnosed as application logic errors.

Specifically: - In languages with threads, default to making components thread-safe with internal locking. Single-threaded-only components should be the exception, not the default, and should assert their constraint at runtime in debug builds. - Protect both read and write paths. Concurrent reads can observe inconsistent state during writes even if writes are serialized — unless reads hold a lock or operate on a snapshot. - Document the concurrency contract in the type or interface, not in a comment. A comment that says “not thread-safe” is read once and forgotten. A lock that must be held is enforced every time. - For data structures with iterators (range scans, cursor-based reads), decide whether iteration produces a snapshot or a live view, document it, and test it under concurrent modification.

4. Recovery must preserve every invariant the normal path maintains

For every invariant your system maintains during normal operation — monotonic counters, sort order, referential integrity, transaction isolation — verify that crash recovery re-establishes that invariant. If recovery can't restore an invariant, the invariant isn't real; it's a fair-weather guarantee that fails exactly when it's needed most.

What goes wrong: The normal write path carefully maintains invariants: monotonically increasing sequence numbers, sorted key order, visibility rules for uncommitted writes. Then a crash happens. Recovery replays the log, rebuilds in-memory state, and reopens for business — but the recovered state violates the invariants the normal path enforced. Aborted transaction data reappears because abort was a status flag, not a physical delete. Monotonic counters reset to zero because they were never persisted. Sort order breaks because recovery doesn't validate it.

Specifically: - For every piece of volatile state that participates in a correctness invariant, either persist it durably or re-derive it correctly during recovery. “It's in memory, it'll be fine” is not a durability strategy. - If abort is implemented as a status change rather than a physical rollback, recovery must replay the abort status — otherwise, aborted writes resurrect and become visible. - Fsync the metadata, not just the data. If your data file is fsynced but your metadata page (root pointer, free list, counters) isn't, a crash can leave the metadata pointing to garbage. Both must be durable, or neither is. - Write a specific test for every invariant: crash the system, recover, and assert the invariant holds. These tests are unpleasant to write and invaluable when they catch bugs.

5. Design repair to match write-path semantics

If writes can create inconsistency through N different mechanisms (conflict resolution strategies, tombstone variants, partial replication, hinted handoffs), the repair mechanism must understand all N. Partial repair creates a false sense of consistency — the system appears healthy by the metrics that repair checks, while divergence accumulates through the mechanisms it doesn't.

What goes wrong: The write path accumulates divergence through multiple mechanisms: sloppy quorums that count hints toward the threshold, conflict resolution split across modules (CRDTs here, last-writer-wins there, custom callbacks elsewhere), tombstones with different semantics at each layer. Anti-entropy repair detects key-range divergence via Merkle trees but reconciles using only one of these mechanisms. The others continue diverging, invisibly.

Specifically: - Audit every mechanism that can create divergence between replicas. For each one, verify that your repair/anti-entropy mechanism handles it. If it doesn't, you have unbounded divergence on that axis. - Unify tombstone semantics across layers. If your storage engine uses physical deletion, your CRDT

layer uses tombstone-with-TTL, and your event log uses soft delete, repair cannot reason about deletes consistently. Pick one model or build explicit translation at each boundary. - Measure divergence, don't just repair it. If you can't answer "how many replicas disagree on how many keys right now," you can't tell whether repair is winning the race against write-path divergence. - Test repair under write load, not just on a quiescent system. Repair that converges when writes are paused but falls behind under production write rates is not a repair mechanism — it's a demo.

6. Include a version field in binary formats from day one

Every binary format you design — wire protocol, storage format, log record, index entry — should include a version field in the first few bytes. The cost is one or two bytes per record. The value is the ability to fix format-level bugs, add integrity mechanisms, and extend the format without a flag-day migration.

What goes wrong: The initial format is designed for the initial requirements. No checksums — we'll add them later. No extensibility — YAGNI. Fixed field widths — simpler to parse. Then a bug is found that requires a format change: corruption that needs a checksum, a field that needs to be wider, a new record type that needs a discriminator. Without a version field, every change requires either a full offline migration or a backward-compatibility hack that accumulates forever.

Specifically: - First byte (or first two bytes after a magic number): format version. Always. Non-negotiable. - Design the format with alignment boundaries that allow resynchronization after corruption. If a single corrupted byte makes the rest of the file unreadable, the format is fragile. Block-aligned records with magic bytes at block boundaries allow recovery to skip the corrupted block and resume. - Include enough redundancy to detect corruption at the record level, not just the file level. A file-level checksum tells you something is wrong; a record-level checksum tells you *what* is wrong and lets you recover everything else. - Plan for the format to outlive the code that writes it. Document the format externally (not just in the serialization code), including byte offsets, endianness, and invariants. Future you — or future someone else — will need to write a recovery tool.

7. Make derived-system consistency prerequisites explicit in the API

If a derived system (materialized view, secondary index, search index, cache) requires specific conditions for consistency — a flush, an old-value capture, a sequence number — enforce those conditions in the API. Implicit consistency contracts become forgotten consistency contracts.

What goes wrong: A CDC-fed secondary index requires two things to stay consistent: (1) the source database must flush changes so CDC consumers can

see them, and (2) change events must include old values so the index can remove stale entries. Neither is enforced. Developers insert rows without flushing, and the index falls behind silently. The CDC layer emits events without old values (it reconstructs them heuristically), and the index accumulates phantom entries. The system appears to work — queries return results — but the results are wrong in ways that are difficult to detect and diagnose.

Specifically: - If a flush or sync is required for correctness, either auto-flush on every mutation (and accept the performance cost) or return a “dirty” token that the caller must resolve before reading from derived systems. Don’t rely on the caller remembering. - If change events must include before-and-after state, make that a requirement of the event schema, not an optional field. Validate it at the producer side. An event without old values should fail loudly, not produce a silently inconsistent downstream. - Provide a consistency check mechanism: a way to verify that a derived system matches its source of truth without rebuilding from scratch. Even a row-count comparison catches gross inconsistencies. - Document the consistency model of each derived system explicitly: is it eventually consistent? Is there a bounded lag guarantee? What happens during a rebuild? Operators need to know what “consistent” means before they can detect inconsistency.

How these rules relate to each other

These seven rules aren’t independent. They form a dependency chain that mirrors the architecture of a data-intensive system:

Rules 1 and 6 (integrity verification and format versioning) are foundation layers. Without them, you can’t detect problems or evolve to fix them. They’re cheap to implement from the start and expensive to retrofit.

Rules 2 and 3 (failure model testing and concurrency enforcement) are verification layers. They determine whether your system actually does what you think it does under the conditions it’ll actually face.

Rules 4 and 5 (recovery invariants and repair semantics) are the consistency layers. They address the hardest class of bugs: the ones where each component is correct but the composition is wrong, typically visible only during failure and recovery.

Rule 7 (explicit derived-system contracts) is the integration layer. It addresses the gap between producing data correctly and consuming it correctly — the last mile where many systems silently lose consistency.

The meta-pattern: **get the boundaries right**. Every rule addresses a boundary — between normal operation and failure, between storage layers, between components, between source and derived data. Components that work in isolation

fail at composition. The bugs that matter most are the ones that live between the things you tested.